UNIVERSIDAD TECNOLÓGICA DE GUTIÉRREZ ZAMORA

PROGRAMA EDUCATIVO:

Ingeniería en Desarrollo y Gestión de Software

NOMBRE DE LA MATERIA:

Desarrollo Móvil Integral

NOMBRE DE LA TAREA:

Unidad II — Seguridad MASVS

Auditoría básica de seguridad con OWASP MASVS / MASTG
(aplicación propia: TotoNinin)

ELABORADO POR:

Héctor Zahid Juárez Simbrón

DOCENTE:

Lothar Yamir Hernández Gaona

1010 - “AEI”

SEPTIEMBRE – DICIEMBRE 2026

Índice

Alcance y controles seleccionados.....3
Procedimiento de prueba.....6
Resultados y evidencias.....8
Hallazgos y nivel de riesgo.....12
Correcciones implementadas.....18
Repetición de pruebas y conclusiones.....20
Entrega y repositorio.....23

Alcance y controles seleccionados

Qué se auditó

La aplicación que audité es TotoNinin, un sistema para gestionar eventos donde los usuarios se registran, ven un calendario con categorías de eventos y dejan comentarios, y un administrador maneja todo desde un panel. Está hecha en PHP 8 con MySQL/MariaDB y se renderiza del lado del servidor, sin framework, y mezcla mysqli procedural con mysqli orientado a objetos. Es una aplicación web: el ZIP del proyecto no trae ningún APK/IPA ni proyecto híbrido (Ionic, Cordova o Capacitor), por eso más abajo explico cómo apliqué MASVS en este caso.

Trabajé con dos versiones. v1-original es el contenido del ZIP tal como lo recibí el 29/10/2026, sin historial de Git previo, y v2-corregida es el mismo código con las 6 correcciones de esta auditoría. Las dos están completas en el repositorio (ver «Entrega y repositorio»). Como referencia usé la versión 2 de OWASP MASVS, con sus grupos STORAGE, CRYPTO, AUTH, NETWORK, PLATFORM, CODE, RESILIENCE y PRIVACY, y la guía MASTG de apoyo para las pruebas.

Las pruebas las hice en un contenedor Linux aislado con PHP 8.3.6 y el servidor integrado (php -S). No pude instalar MariaDB/MySQL porque el entorno no tenía acceso a los paquetes del sistema, así que usé una base de datos de prueba en SQLite con una capa que traduce las mismas llamadas mysqli que usa la app (pruebas/mysqli_shim.php). Esa capa no agrega ni quita ninguna lógica de seguridad, y además revisé cada hallazgo leyendo directamente el código PHP real.

Reglas de trabajo

Solo audité mi propia aplicación y no ataqué ningún servicio externo. Todos los datos son de prueba: usuarios ficticios con correos @totoninin.test y claves marcadas como Test/Prueba (por ejemplo admin@totoninin.test y user@totoninin.test), sin ningún dato personal real. Las claves, los hashes y las

cookies de sesión aparecen ocultos en todas las capturas y solo se deja su formato. Antes de corregir guardé una copia completa de la versión original (v1-original) como respaldo, y los cambios quedaron registrados en el historial de Git del repositorio.

Aclaración de alcance: MASVS aplicado a una aplicación web

MASVS está pensado para aplicaciones móviles, nativas o híbridas, que se distribuyen como binario (APK/IPA). TotoNinin es una aplicación web en PHP que se ejecuta en el servidor y no trae ningún archivo móvil compilado. Aun así, las diapositivas 50 y 51 de la unidad incluyen criterios que no dependen de la plataforma, como "HTTP o TLS mal validado" y "Secretos incluidos en Git", así que apliqué los grupos de MASVS que tienen un equivalente en una aplicación cliente-servidor: STORAGE, CRYPTO, AUTH, NETWORK, CODE y PLATFORM (esta última solo en el control de acceso a las partes de administración y en la validación de archivos).

Los controles que dependen de un binario instalado en el celular los marqué como No aplica. Son RESILIENCE (evitar que modifiquen o descompilen la app) y la parte de PLATFORM sobre permisos del sistema, WebViews y enlaces profundos. La razón es que el código de TotoNinin nunca llega al dispositivo del atacante porque corre en el servidor. La justificación completa está en el «Checklist MASVS».

Controles seleccionados

Esta tabla muestra qué pasó con cada grupo de MASVS:

Grupo MASVS	Resultado del alcance
MASVS-STORAGE	Evaluado
MASVS-CRYPTO	Evaluado
MASVS-AUTH	Evaluado
MASVS-NETWORK	No aplica (justificado), ver «Checklist MASVS»

Grupo MASVS	Resultado del alcance
MASVS-PLATFORM	Evaluado en parte (control de acceso y validación de archivos); el resto no aplica, ver «Checklist MASVS»
MASVS-CODE	Evaluado
MASVS-RESILIENCE	No aplica (justificado), ver «Checklist MASVS»
MASVS-PRIVACY	Evaluado (cumple, con una observación menor que no bloquea)

Procedimiento de prueba

Seguí las ocho fases de la metodología de la unidad: inventariar, inspección estática, prueba dinámica, registrar, priorizar, corregir, reprobar y entregar.

Inventario

Primero anoté qué maneja la app. En la base de datos hay cuatro tablas: users (nombre, correo y hash de contraseña), events (título, descripción, fecha, foto y tipo), comments (texto libre ligado a un evento) y event_type (catálogo de categorías). Las funciones delicadas son iniciar sesión, el registro público, el alta, edición y eliminación de usuarios, eventos y categorías desde el panel de administración, la subida de fotos, los comentarios y cerrar sesión.

En el código original hay dos niveles de permisos que no están escritos en ningún lado: usuario autenticado y administrador, y este último se decide comparando el correo con un valor fijo, sin tabla de roles. Sin iniciar sesión se puede entrar a index.php, paginas/registro.php, paginas/login.php, paginas/informacion.php y paginas/preguntas.php.

Inspección estática

Revisé a mano todo el código PHP (2,673 líneas en 37 archivos) buscando entradas del usuario pegadas dentro de consultas SQL, credenciales escritas en el código, el algoritmo de hash de las contraseñas, la validación de los archivos subidos, la configuración de las cookies de sesión, el manejo de errores y si existía un .gitignore. Después comparé línea por línea (diff) v1-original contra v2-corregida para confirmar que cada corrección es pequeña y no cambia nada que no tenga que ver con el hallazgo.

Prueba dinámica

Levanté la aplicación con el servidor integrado de PHP (php -S) sobre la base de datos de prueba y ejecuté un script de pruebas HTTP automático que se puede repetir (pruebas/run_tests.py, también en el repositorio). Para cada versión del código, el script deja la base de datos en un estado conocido antes de cada caso para que las pruebas no se afecten entre sí y manda peticiones HTTP reales, con y sin sesión, con datos maliciosos y con datos normales. Guarda lo que pasó (código de estado, cabeceras importantes y partes de la respuesta) ocultando automáticamente las cookies de sesión y los hashes. Al final compara el resultado con el criterio de cada hallazgo y marca el caso como SEGURO/CUMPLE o VULNERABLE/NO CUMPLE.

En total fueron 24 casos: 19 de seguridad, repartidos en los 6 hallazgos, y 5 de regresión (login normal, calendario, panel de administración, registro público y página de inicio) para comprobar que las correcciones no rompieran nada que ya funcionaba.

Registrar y priorizar

Cada hallazgo tiene su ficha con ID, grupo MASVS, ubicación, forma de reproducirlo, evidencia, probabilidad, impacto y nivel de riesgo. Para el nivel usé la escala de la unidad: riesgo = impacto × probabilidad, con valores del 1 al 3 en cada uno, donde 1–3 es Bajo, 4–6 Medio, 7–8 Alto y 9 Crítico.

Resultados y evidencias

Esta tabla compara los 24 casos de prueba ejecutados en las dos versiones del código:

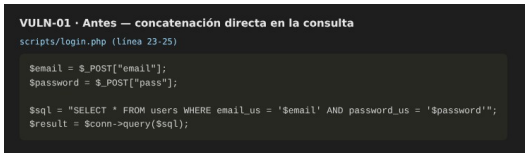
Versión	Casos seguros / cumplen	Casos vulnerables	Regresión funcional
v1-original	**8 / 24**	16 de 19 casos de seguridad	5 / 5 (ya funcionaba)
v2-corregida	**24 / 24**	0 de 19 casos de seguridad	5 / 5 (sin regresiones)

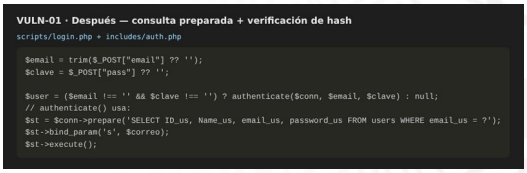
Evidencia por hallazgo (código y comportamiento, antes y después)

*Las claves, los hashes y las cookies de sesión están ocultos con **** en todas las capturas; solo se deja su formato para poder comprobar el control.*

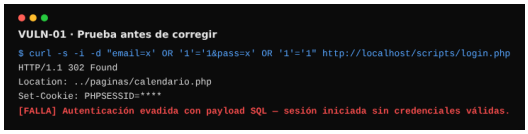
VULN-01 · Inyección SQL

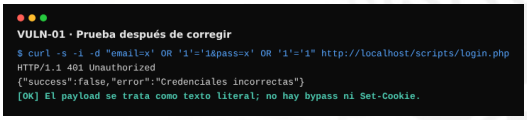
Código antes (izquierda) y después (derecha):





Lo que pasó en las pruebas antes (izquierda) y después (derecha):





VULN-02 · Control de acceso roto

Código antes (izquierda) y después (derecha):

VULN-02 · Antes — sin verificación de rol

scripts/eliminar_usuario.php (línea 1-16)

```php
<?php
session_start();

include "conexion.php";

if(empty($_GET['id'])){
    header("location: lista_usuarios.php");
} else {
    $id_usuario = $_GET['id'];
    ...
    $query_delete = mysqli_query($conection, "DELETE FROM users WHERE ID_us = $id_usuario");
}
```

VULN-02 · Después — control de acceso en el servidor

scripts/eliminar_usuario.php + includes/auth.php

```php
<?php
require_once __DIR__ . '/../includes/auth.php';
require_admin(); // VULN-02: exige sesión + rol administrador

function require_admin() {
    require_login();
    if (empty($_SESSION['es_admin'])) {
        http_response_code(403);
        exit('Acceso denegado');
    }
}
```

Lo que pasó en las pruebas antes (izquierda) y después (derecha):

VULN-02 · Prueba antes de corregir (sin sesión)

```
$ curl -s -i -X POST http://localhost/scripts/eliminar_usuario.php?id=3 -d id=3
HTTP/1.1 302 Found
Location: lista_usuarios.php
[FALLA] El usuario id=3 fue eliminado por un visitante sin sesión iniciada.
```

VULN-02 · Prueba después de corregir (sin sesión)

```
$ curl -s -i -X POST http://localhost/scripts/eliminar_usuario.php?id=3 -d id=3
HTTP/1.1 302 Found
Location: ../paginas/login.php
[OK] Redirigido al login; el registro permanece intacto en la base de datos.
```

VULN-03 · Contraseñas guardadas con MD5

Código antes (izquierda) y después (derecha):

VULN-03 · Antes — MD5 sin sal

scripts/registro_usuario.php (línea 17)

```php
$Name_us = $_POST['Name_us'];
$email_us = $_POST['email_us'];
$password_us = md5($_POST['password_us']); // sin sal, reversible por diccionario

$query_insert = mysqli_query($conection,
  "INSERT INTO users(Name_us,email_us,password_us) VALUES('$Name_us','$email_us','$password_us')");
```

VULN-03 · Después — bcrypt con sal + migración de cuentas antiguas

scripts/registro_usuario.php + includes/auth.php

```php
$password_us = password_hash($_POST['password_us'], PASSWORD_DEFAULT); // bcrypt con sal

$ins = $conection->prepare("INSERT INTO users(Name_us,email_us,password_us) VALUES(?,?,?)");
$ins->bind_param("sss", $Name_us, $email_us, $password_us);

// En el login, si la cuenta aún tiene un hash MD5 heredado, se valida una vez y se migra:
elseif (preg_match('/^[a-f0-9]{32}$/i', $hash) && hash_equals(strtolower($hash), md5($clave))) {
    $ok = true; $migrar = true;   // se reemplaza por bcrypt en la misma petición
}
```

Lo que pasó en las pruebas antes (izquierda) y después (derecha):

VULN-03 · Prueba antes de corregir

```
$ sqlite3 totoninin.db "SELECT password_us FROM users WHERE email_us='nuevo@totoninin.test'"
[hash MD5 de 32 caracteres — oculto]
[FALLA] Hash MD5 sin sal: crackeable con diccionario/rainbow table en segundos.
```

VULN-03 · Prueba después de corregir

```
$ sqlite3 totoninin.db "SELECT password_us FROM users WHERE email_us='nuevo@totoninin.test'"
[hash bcrypt $2y$10$... — oculto]
[OK] bcrypt con sal y factor de costo; no es reversible por diccionario.
```

VULN-04 · Subida de archivos sin validar

Código antes (izquierda) y después (derecha):

VULN-04 · Antes — sin validar tipo ni contenido

scripts/registro_evento.php (línea 39-48)

```php
// Verificar si la foto es válida
if ($foto['error'] !== 0) {
    echo "Error: Foto inválida";
    exit;
}

// Mover la foto a una carpeta temporal
$rutaTemporal = $foto['tmp_name'];
$rutaDestino = '../fotos/' . $foto['name'];   // nombre y extensión del cliente, sin validar
move_uploaded_file($rutaTemporal, $rutaDestino);
```

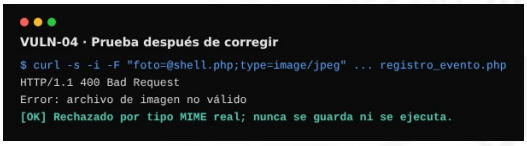
VULN-04 · Después — valida MIME real y bloquea ejecución

includes/upload.php + fotos/.htaccess

```php
function guardar_foto_evento($f) {
    if (!is_uploaded_file($f['tmp_name']) || $f['size'] > 2*1024*1024) return null;
    $permitidos = ['image/jpeg'=>'jpg','image/png'=>'png','image/gif'=>'gif','image/webp'=>'webp'];
    $mime = (new finfo(FILEINFO_MIME_TYPE))->file($f['tmp_name']);     // MIME real, no el del navegador
    if (!isset($permitidos[$mime]) || @getimagesize($f['tmp_name']) === false) return null;
    $nombre = bin2hex(random_bytes(16)) . '.' . $permitidos[$mime];    // nombre aleatorio, no el del cliente
    move_uploaded_file($f['tmp_name'], __DIR__.'/../fotos/'.$nombre);
    return '../fotos/' . $nombre;
}
# fotos/.htaccess: php_flag engine off  (defensa en profundidad)
```

Lo que pasó en las pruebas antes (izquierda) y después (derecha):





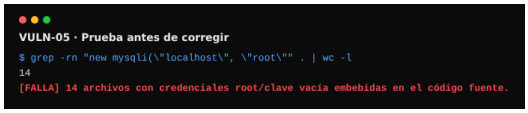
VULN-05 · Credenciales escritas en el código

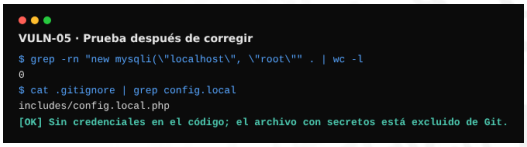
Código antes (izquierda) y después (derecha):





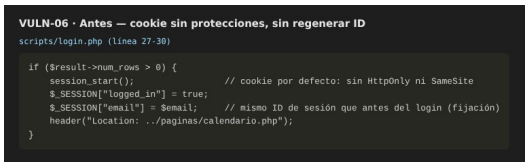
Lo que pasó en las pruebas antes (izquierda) y después (derecha):

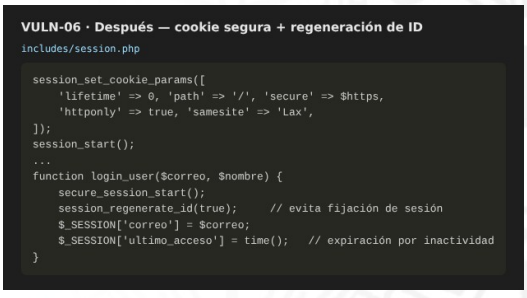




VULN-06 · Manejo inseguro de la sesión

Código antes (izquierda) y después (derecha):





Lo que pasó en las pruebas antes (izquierda) y después (derecha):


VULN-06 · Prueba antes de corregir
$ curl -s -i -d "correo=user@totoninin.test&password=UserTest#2026" http://localhost/paginas/login.php
Set-Cookie: PHPSESSID=***; path=/
[FALLA] Cookie sin HttpOnly ni SameSite; además el ID de sesión no se regenera (fijación).


VULN-06 · Prueba después de corregir
$ curl -s -i -d "correo=user@totoninin.test&password=UserTest#2026" http://localhost/paginas/login.php
Set-Cookie: PHPSESSID=***; path=/; HttpOnly; SameSite=Lax
[OK] Cookie protegida; el ID de sesión se regenera en cada login (sin fijación).

Hallazgos y nivel de riesgo

Cada hallazgo tiene su ficha. El riesgo es impacto × probabilidad (de 1 a 3 cada uno; 1–3 Bajo, 4–6 Medio, 7–8 Alto, 9 Crítico). Es la escala de la unidad, no una escala oficial de OWASP MASVS.

VULN-01 — Inyección SQL

Grupo MASVS	MASVS-CODE (principal) · MASVS-AUTH (secundario)
Ubicación	scripts/login.php (consulta del login); scripts/load_events.php y scripts/obtener_eventos.php (parámetro "mes"); scripts/lista_usuarios.php, lista_eventos.php y lista_categoria.php (parámetro "pagina"); scripts/registro_usuario.php, editar_usuario.php, registro_categoria.php y editar_categoria.php (correo y descripción)
Evidencia	Casos P-01a / P-01b; ver «Evidencia por hallazgo» y la carpeta pruebas/ del repositorio

Para comprobarlo mandé un POST a scripts/login.php con x' OR '1'='1 tanto en el correo como en la contraseña. El código pega lo que escribe el usuario directo dentro de la consulta SQL, así que la condición siempre sale verdadera y la app me dejó entrar sin conocer ninguna contraseña. Ya dentro con cualquier usuario, abrí scripts/load_events.php?mes=0 UNION SELECT ID_us,Name_us,password_us,... FROM users -- y esa página, que solo debería listar los eventos del mes, me mostró los hashes de contraseña de todos los usuarios.

Probabilidad	Impacto	Riesgo
3 (alta)	3 (alto)	**Crítico**

Para corregirlo cambié todas las consultas que armaban el SQL con lo que escribe el usuario por consultas preparadas (prepare y bind_param de mysqli). Además, los valores que deben ser números, como mes, pagina e id, ahora se convierten a entero antes de usarlos. El login quedó en una sola función, authenticate(), dentro de includes/auth.php.

VULN-02 — Control de acceso roto en las funciones de administrador

Grupo MASVS	MASVS-AUTH (principal) · MASVS-PLATFORM (secundario)
Ubicación	scripts/lista_usuarios.php, eliminar_usuario.php, editar_usuario.php, registro_usuario.php, lista_categoria.php, eliminar_categoria.php, registro_categoria.php, editar_categoria.php, lista_eventos.php, eliminar_confirmar_evento.php, registro_evento.php, editar_evento.php, paginas/admin.php
Evidencia	Casos P-02a / P-02b / P-02c / P-02d; ver «Evidencia por hallazgo» y la carpeta pruebas/ del repositorio

Sin iniciar sesión pedí scripts/lista_usuarios.php y me mostró la lista completa de usuarios con nombre y correo. Con un POST a scripts/eliminar_usuario.php?id=3, también sin sesión, el usuario se borró de la base de datos. Con otro POST a scripts/editar_usuario.php?id=2 le cambié la contraseña a otro usuario y después pude entrar a su cuenta con la clave que puse yo. Por último, entré con un usuario normal (no administrador) y el panel de administración se abrió igual, porque ningún script revisa el rol.

Probabilidad	Impacto	Riesgo
3 (alta)	3 (alto)	**Crítico**

Creé includes/auth.php con las funciones require_login() y require_admin(). Los 14 scripts de administración ahora piden sesión iniciada y rol de administrador antes de leer o escribir cualquier dato. El rol (es_admin) se calcula en el servidor a partir de ADMIN_EMAIL.

VULN-03 — Contraseñas guardadas con MD5

Grupo MASVS	MASVS-CRYPTO (principal) · MASVS-STORAGE (secundario)
Ubicación	scripts/registro.php y scripts/registro_usuario.php (alta de usuarios desde el panel de administración)
Evidencia	Casos P-03a / P-03b; ver «Evidencia por hallazgo» y la carpeta pruebas/ del repositorio

Registré un usuario desde scripts/registro_usuario.php y revisé la columna password_us en la base de datos. El valor era un hash MD5 de 32 caracteres

hexadecimales y sin sal, que se puede descifrar en segundos con un diccionario si la base de datos llegara a filtrarse. El formulario público de registro (paginas/registro.php) ya usaba password_hash() con bcrypt desde el original, así que el problema estaba solo en el alta que hace el administrador y en la clase Registro, que quedó de una versión anterior.

Probabilidad	Impacto	Riesgo
2 (media)	2 (medio)	Medio

Cambié md5() por password_hash($clave, PASSWORD_DEFAULT), que usa bcrypt, en todos los lugares donde se crea o se edita una contraseña. También agregué una migración: si una cuenta vieja todavía tiene hash MD5, en su primer login correcto se valida una vez y se guarda de nuevo con bcrypt.

VULN-04 — Subida de archivos sin validar (ejecución de código en el servidor)

Grupo MASVS	MASVS-CODE (principal) · MASVS-PLATFORM (secundario)
Ubicación	scripts/registro_evento.php y scripts/editar_evento.php (campo "foto")
Evidencia	Casos P-04a / P-04b / P-04c; ver «Evidencia por hallazgo» y la carpeta pruebas/ del repositorio

Con una cuenta normal, creada desde el registro público, creé un evento y en la foto subí un archivo shell.php que contiene <?php echo 'RCE_'.(6*7); ?>. La aplicación solo revisa $foto['error'] y mueve el archivo a la carpeta fotos/ con el mismo nombre y extensión que manda el navegador, sin comprobar qué contiene en realidad. Al abrir fotos/shell.php directamente, el servidor lo ejecutó como PHP y respondió RCE_42, o sea que cualquiera podría correr código en el servidor con los permisos del servidor web.

Probabilidad	Impacto	Riesgo
3 (alta)	3 (alto)	Crítico

Agregué includes/upload.php con la función guardar_foto_evento(). Revisa que el archivo venga de una subida real (is_uploaded_file), limita el tamaño, comprueba el tipo real con fileinfo en lugar del que dice el navegador y lo compara con una lista de imágenes permitidas. Después confirma con getimagesize() que de verdad sea una imagen y la guarda con un nombre aleatorio y la extensión que corresponde a su tipo, nunca la que mandó el cliente. Como protección extra puse un fotos/.htaccess que apaga la ejecución de PHP en esa carpeta.

VULN-05 — Credenciales escritas dentro del código

Grupo MASVS	MASVS-STORAGE (principal) · MASVS-CODE (secundario)
Ubicación	scripts/conexion.php, bd_php/conexion.php, scripts/login.php, scripts/add_comment.php, scripts/evento_detalle.php, scripts/registro_evento.php, scripts/editar_evento.php, paginas/login.php, paginas/registro.php, index.php, scripts/load_events.php, scripts/obtener_eventos.php, scripts/eventos.php, scripts/registro.php, bd_php/usuarioBD.php
Evidencia	Casos P-05a / P-05b / P-05c; ver «Evidencia por hallazgo» y la carpeta pruebas/ del repositorio

Busqué en el código la cadena new mysqli("localhost","root","",...) y aparece repetida en 14 archivos, con el usuario root y la contraseña vacía escritos directo. Además bd_php/usuarioBD.php tenía error_reporting(E_ALL) e ini_set("display_errors",1), y mostraba el mensaje de error de MySQL al usuario con die("Error de conexión: " . $e->getMessage()), lo que deja ver rutas, nombres de tablas y la versión de MySQL a cualquiera que visite la página. El proyecto tampoco tenía archivo .gitignore, así que si algún día se creaba un archivo de configuración con credenciales reales, se habría subido al repositorio sin ningún control.

Probabilidad	Impacto	Riesgo
2 (media)	2 (medio)	**Medio**

Dejé la conexión en includes/db.php e includes/config.php, que leen el host, el usuario, la clave y la base de datos de variables de entorno o del archivo includes/config.local.php, que no se sube al repositorio. Los errores de conexión y

de consulta ahora se guardan con error_log() y al usuario solo le sale un mensaje general. También agregué un .gitignore que excluye config.local.php, los archivos *.env y los registros temporales de las pruebas.

VULN-06 — Manejo inseguro de la sesión

Grupo MASVS	MASVS-AUTH (principal)
Ubicación	scripts/login.php, paginas/login.php, scripts/cerrar_sesion.php, includes/header.php, includes/nav.php
Evidencia	Casos P-06a / P-06b / P-06c / P-06d; ver «Evidencia por hallazgo» y la carpeta pruebas/ del repositorio

Al iniciar sesión, la cabecera Set-Cookie del PHPSESSID no trae HttpOnly ni SameSite, y el identificador de sesión es el mismo que tenía el visitante antes de entrar porque no se llama a session_regenerate_id(). Eso permite que un atacante le dé a la víctima un identificador que él ya conoce, por ejemplo con un enlace, y lo siga usando cuando la víctima inicie sesión (fijación de sesión). Otro problema es que el login responde distinto cuando el correo no existe y cuando la contraseña es incorrecta, así que se pueden ir probando correos para saber cuáles están registrados. Además, scripts/cerrar_sesion.php llama a session_destroy() pero nunca borra la cookie del navegador, por lo que el identificador sigue viajando en cada petición. Por último, el rol de administrador se decide comparando el correo con un valor fijo dentro de includes/nav.php, repetido en varios archivos, en lugar de calcularlo una sola vez en el servidor al autenticar.

Probabilidad	Impacto	Riesgo
2 (media)	2 (medio)	**Medio**

Creé includes/session.php. La cookie de sesión ahora lleva HttpOnly y SameSite=Lax, y Secure cuando se usa HTTPS; la sesión expira a los 30 minutos sin actividad; se llama a session_regenerate_id(true) en cada login; y logout_user() vacía $_SESSION, destruye la sesión en el servidor y expira la cookie en el navegador. El login ahora siempre responde "Credenciales incorrectas" sin decir

cuál fue el motivo, y el rol es_admin se calcula una sola vez en el servidor al autenticar.

Correcciones implementadas

Aquí está lo que cambió entre v1-original y v2-corregida. El detalle línea por línea está como diff en el repositorio, en el commit "Correcciones VULN-01..06 (OWASP MASVS) + evidencias + reporte técnico".

Archivos nuevos

Archivo	Para qué sirve
includes/config.php	Lee la configuración (host, usuario y clave de la base de datos, correo del administrador) de variables de entorno o de includes/config.local.php, en lugar de tenerla escrita en el código.
includes/ config.local.example.php	Ejemplo para que cada entorno (desarrollo o producción) cree su propio includes/config.local.php, que no se sube a Git.
includes/db.php	Función db(), que entrega la conexión mysqli ya configurada. Si falla, guarda el detalle en el log del servidor y al usuario le muestra un mensaje general (VULN-05).
includes/session.php	secure_session_start() (cookie con HttpOnly y SameSite, expiración por inactividad), login_user() (cambia el ID de sesión) y logout_user() (destruye la sesión y expira la cookie). Corrige VULN-06.
includes/auth.php	require_login() y require_admin() (VULN-02) y authenticate(), con consulta preparada, password_verify() y migración de hashes MD5 viejos a bcrypt (VULN-01 y VULN-03).
includes/upload.php	guardar_foto_evento(): revisa el tipo real, el tamaño y que sea una imagen válida, y genera un nombre de archivo aleatorio (VULN-04).
fotos/.htaccess	Apaga la ejecución de PHP dentro de la carpeta de fotos subidas, como protección extra para VULN-04.
.gitignore	Evita subir a Git includes/config.local.php, los archivos *.env y los archivos temporales de prueba (VULN-05).

Archivos modificados

Archivo	Qué se corrigió
scripts/login.php, paginas/login.php, paginas/login.js	VULN-01 (consulta preparada) y VULN-06 (sesión segura, mensaje general y textContent en lugar de innerHTML)

Archivo	Qué se corrigió
scripts/conexion.php, bd_php/conexion.php, index.php, scripts/{load_events,obtener_eventos,evento_detalle,add_comment,eventos,registro}.php, paginas/registro.php, bd_php/usuarioBD.php	VULN-05 (credenciales fuera del código, errores sin detalle para el usuario)
scripts/ {registro_usuario,editar_usuario, registro_categoria,editar_categoria,eliminar_usuario,eliminar_categoria,eliminar_confirmar_evento,lista_usuarios,lista_categoria,lista_eventos,buscar_usuario}.php, paginas/admin.php	VULN-02 (require_admin) y VULN-01 (consultas preparadas y parámetros numéricos convertidos a int)
scripts/registro_evento.php, scripts/editar_evento.php	VULN-04 (validación real del archivo con includes/upload.php)
scripts/cerrar_sesion.php, includes/header.php, includes/nav.php	VULN-06 (logout_user() y rol es_admin calculado en el servidor)

Repetición de pruebas y conclusiones

Reprueba

Corrí otra vez las mismas 24 pruebas automáticas contra v2-corregida y 24 de 24 casos salieron SEGURO/CUMPLE, incluidos los 5 de regresión (login normal, calendario de eventos, panel de administración, registro público y página de inicio). Con esto se ve que las correcciones quitan los 6 hallazgos sin romper ninguna función de la aplicación.

Pendientes y recomendaciones

Quedaron cosas que no corregí. NETWORK no se pudo evaluar porque el proyecto no trae la configuración del servidor de producción; ahí habría que forzar HTTPS y agregar la cabecera Strict-Transport-Security. Tampoco puse un límite de intentos de login (rate limiting), que ayudaría contra la fuerza bruta aunque las contraseñas ya estén en bcrypt.

De las cabeceras de seguridad solo agregué X-Content-Type-Options, X-Frame-Options y Referrer-Policy, y no puse Content-Security-Policy para no romper las fuentes e íconos externos que usa la app. La observación de PRIVACY (falta un aviso de privacidad y una opción para que el usuario borre su cuenta) la dejé como mejora futura. Por último, en un servidor real la base de datos debe usar un usuario de MySQL con permisos mínimos y una contraseña fuerte, nunca root sin clave; includes/config.local.example.php ya muestra ese cambio.

Conclusiones

Encontré 6 fallas, una más que el mínimo de 5 que pedía la tarea: 3 de riesgo Crítico (inyección SQL, control de acceso roto y subida de archivos sin validar) y 3 de riesgo Medio (MD5 en las contraseñas, credenciales escritas en el código y manejo inseguro de la sesión). Las seis quedaron corregidas y las comprobé con pruebas

que se pueden repetir, antes y después del cambio. Las 5 pruebas de regresión confirman que el inicio de sesión, el calendario, el panel de administración y el registro público siguen funcionando igual que antes.

RESILIENCE y la parte de PLATFORM que depende de permisos del sistema los dejé como No aplica porque TotoNinin es una aplicación web sin empaquetado móvil. La justificación está en el checklist de abajo.

Checklist MASVS

Área	Qué se revisó	Estado	Evidencia o justificación
STORAGE	Credenciales escritas en el código, manejo de secretos, errores con demasiado detalle y secretos en Git.	NC	VULN-05 (P-05a/b/c). Corregido y comprobado en la reprueba.
CRYPTO	Algoritmo de hash de las contraseñas y nombres de archivo o identificadores.	NC	VULN-03 (P-03a/b): de MD5 a bcrypt. Corregido.
AUTH	Inyección SQL en el login, acceso a las funciones de administrador, cookie de sesión, fijación de sesión, saber qué correos existen y cierre de sesión.	NC	VULN-01, VULN-02 y VULN-06. Corregidos; ver sus fichas y la reprueba.
NETWORK	Transporte entre cliente y servidor (HTTP/TLS).	N/A	El proyecto no incluye la configuración del servidor web (vhost de Apache o Nginx) ni un certificado de producción. Tampoco encontré en el código endpoints propios con http://, solo recursos externos como fuentes e íconos. Por eso queda fuera de lo que se puede evaluar con el código; en producción habría que forzar HTTPS y usar la cabecera Strict-Transport-Security.
PLATFORM	Validación de archivos subidos (lo más parecido en web a permisos e IPC) y control de acceso a las partes de administración.	NC	VULN-04 (subida de archivos) y VULN-02 (acceso a los paneles). Corregidos. No aplican WebViews, enlaces profundos ni permisos del sistema operativo porque la app es web, se renderiza en el servidor y no tiene APK/IPA.

Área	Qué se revisó	Estado	Evidencia o justificación
CODE	Validación de entradas, manejo de errores y modo de depuración.	NC	VULN-01 (inyección SQL) y VULN-05 (display_errors). Corregidos.
RESILIENCE	Manipulación, ingeniería inversa y controles del entorno de ejecución.	N/A	No aplica. La app no se distribuye como binario (APK/IPA) sino como código PHP que se interpreta en el servidor, así que no hay un archivo en el celular que alguien pueda descompilar o modificar. El código del servidor nunca llega al dispositivo del usuario, y esa es la idea de fondo de los controles de resistencia de MASVS.
PRIVACY	Datos mínimos, consentimiento, rastreadores y retención.	C	Los datos que pide la app (nombre, correo, contraseña y comentarios) son los necesarios para funcionar y no encontré scripts de rastreo o analítica de terceros. Una observación menor que no bloquea: no hay aviso de privacidad ni opción para eliminar la cuenta propia; lo dejo como recomendación para después, fuera de las 6 correcciones de esta auditoría.

C = cumple · NC = no cumplía (se corrigió, se volvió a probar y ahora cumple) · N/A = no aplica (justificado en la tabla)

Entrega y repositorio

Subí todo al repositorio de GitHub https://github.com/hectorjua28/Seguridad-Practica-. Tiene la carpeta v1-original/ con la aplicación tal como la recibí, v2-corregida/ con las 6 correcciones, pruebas/ con run_tests.py, la capa de compatibilidad mysqli_shim.php y los resultados de cada ejecución (resultados.txt y resultados.json) de las dos versiones, evidencias/ con las capturas de este reporte y reporte/ con este documento.

El historial de Git empieza con un commit que guarda v1-original sin cambios y sigue con el commit de las correcciones VULN-01..06 y las evidencias, así que se puede comparar el antes y el después con git diff.